

MEDIDOR DE CAPACITANCIA CON ARDUINO Y CI 555

(05 de junio del 2026)

Yahir Alejandro Baeza Collazo

ya.baezacollazo@ugto.mx

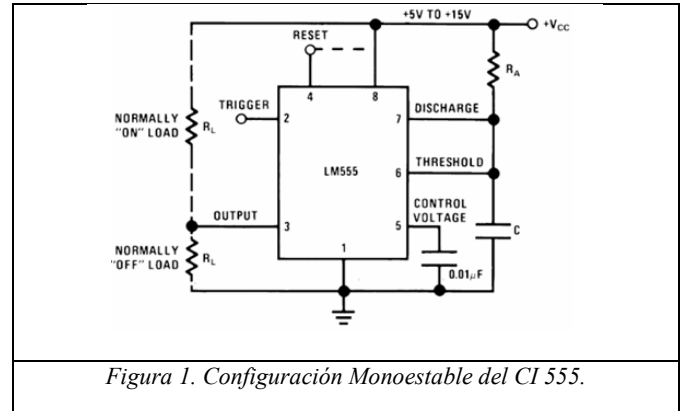
Resumen -En el siguiente reporte se presenta el funcionamiento de un medidor de capacitancia en nano faradios y microfaradios, con un rango de medición de 0.15 nF a 1000 mF . La medición se basa en las características del circuito integrado 555, configurado como multivibrador monoestable, el procesador de datos utilizado fue el Arduino Nano, finalmente el multiplexor CD4051 realiza una función de auto rango controlada por el Arduino.

I. INTRODUCCIÓN

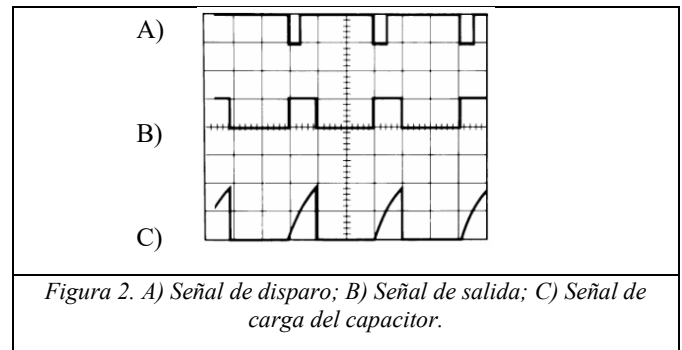
El método utilizado en el presente proyecto para medir la capacitancia con el microcontrolador Arduino se basó en calcular una constante de tiempo de carga en el capacitor. El circuito integrado 555 en la configuración monoestable produce un pulso de salida cuya duración puede determinarse mediante el valor de la resistencia y condensador externo, la Ecuación 1 describe el tiempo de carga del capacitor en la configuración monoestable.

$$T = 1.1 \times R_1 \times C_1 \quad (1)$$

Esta configuración se observa en la Figura 1, el capacitor externo (prueba) se mantiene inicialmente descargado por un transistor interno que parte del pin 7 a tierra. Al aplicar un pulso negativo en el pin 2 de un valor menor que $\frac{1}{3}V_{cc}$, el comparador interno del circuito integrado cambia su estado de salida activando el flip-flop, el cual libera el cortocircuito a través del condensador y la salida del pin 3 se pone en alto [4].



El voltaje a través del capacitor se incrementa exponencialmente durante un periodo de tiempo dado por la Ecuación 1, cuando el voltaje en el capacitor es igual a $\frac{2}{3}V_{cc}$ el comparador encargado de monitorear el voltaje del capacitor por el pin 6, cambia su estado de salida y desactiva el flip-flop, provocando que el capacitor se descargue y el estado de la salida cambie a bajo. El condensador conectado al pin 5, mejora la estabilidad en el voltaje de referencia que contienen los comparadores internos del circuito integrado. En la Figura 2 se observan las formas de onda de diferentes componentes del circuito [4].



Como se observa en la Figura 2, el periodo en alto de la señal de salida coincide con el periodo de carga y descarga del

capacitor, describiendo Ecuación 1 de la siguiente manera se puede determinar la capacitancia (C_1),

$$C_1 = \frac{T}{1.1 \times R_1} \quad (2)$$

El periodo de tiempo de la señal de salida se medirá por medio del microcontrolador Arduino Nano, además se programará la Ecuación 2 para obtener el valor resultante de la capacitancia. Una de las limitantes de medir la capacitancia con el circuito integrado 555 es el rango de medición, ya que si se tiene una resistencia de un valor grande y un capacitor de igual manera, la constante de tiempo será grande y el cálculo de la capacitancia tardara demasiado tiempo, por otro lado si la resistencia es de un valor pequeño y el capacitor de igual manera, la constante de tiempo será muy pequeña y tendrá un gran margen de error, para solucionar este inconveniente se utilizará un multiplexor CD4051. El multiplexor contiene 3 pines de control binarios llamados A, B, C. Estos pines en conjunto forman el numero binario de cada una de las salidas/entradas, es decir que el menor número debe ser 000 = 0 y el mayor 111 = 7, por lo tanto, se tienen 8 salidas/entradas como máximo [5].

El multiplexor contiene un pin *INH* (inhibit) que se encarga de habilitarlo o deshabilitarlo, normalmente va conectado a GND para que el multiplexor funcione, el pin *COM* es el canal común principal, la entrada o salida que se conecte a este pin saldrá en la salida/entrada que se habilite por medio de los pines A, B, C. El pin V_{EE} es el voltaje de suministro negativo y va conectado a GND para señales unipolares, V_{SS} es tierra (GND), finalmente V_{DD} es el voltaje de suministro positivo (5V) [5].

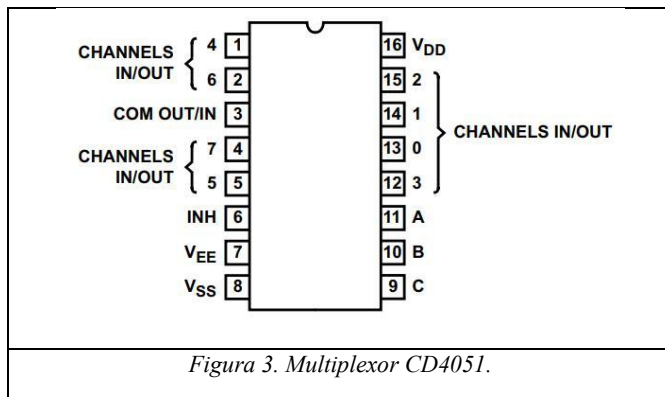


Figura 3. Multiplexor CD4051.

El multiplexor CD4051 contiene una resistencia interna que varía desde los 400 Ω a 520 Ω con una alimentación de 5 V a una temperatura de 25 $^{\circ}\text{C}$ [5].

II. OBJETIVOS

El objetivo principal de este proyecto fue obtener la medición de diferentes capacitores utilizando un microcontrolador como Arduino, un circuito integrado 555 y un multiplexor CD4051 para cambiar automáticamente los rangos de medición. El procesamiento se basó en el cálculo de una constante de tiempo de la carga y descarga del capacitor. El rango de medición se encuentra entre los 0.15 *nf* y los 1000 *uf*. Cabe mencionar que el circuito de medición se calibro para obtener un margen de medición mínimo.

III. MARCO TEÓRICO

Capacitancia: capacidad de un componente o circuito para recoger y almacenar energía en forma de carga eléctrica.

Periodo: espacio de tiempo que tarda un ciclo completo de la onda en repetirse.

Constante de tiempo: medida que indica la velocidad de respuesta de un sistema ante un cambio o perturbación.

Multiplexor: dispositivo que permite seleccionar una señal específica de entre varias entradas y transmitirla a través de una única salida.

Circuito Integrado: chip de material semiconductor que contiene miles o millones de componentes electrónicos interconectados como transistores, resistencias y diodos.

Resistencia interna: oposición que presentan los materiales semiconductores y los componentes internos al flujo de la corriente eléctrica.

Flip-flop: circuito electrónico digital que funciona como la unidad básica de memoria. Puede almacenar un bit de información (0 o 1) y mantener ese estado de forma indefinida hasta que la señal externa conmute su estado.

Capacitor: dispositivo eléctrico capaz de almacenar energía a través de un campo eléctrico.

Resistencia: componente eléctrico que se opone al flujo de corriente eléctrica en un circuito, limitando la cantidad de electricidad que pasa a través de él.

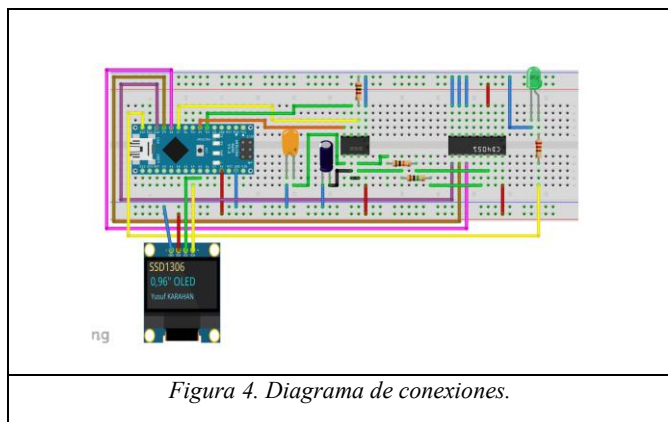
IV. MATERIALES

- Arduino nano
- Circuito integrado 555
- Multiplexor CD4051
- Resistencia de $1\text{ K}\Omega$
- Resistencia de $1\text{ M}\Omega$
- Capacitor de $0.01\text{ }\mu\text{f}$
- Jumpers
- Protoboard
- Placa perforada
- Pantalla oled ssd1306
- Cable de datos para Arduino Nano
- Computadora
- Capacitores de diferentes valores

V. DESARROLLO

A. Diagrama de conexiones.

El primer paso para realizar el circuito físico fue diseñar las conexiones por medio de un software en este caso se utilizó Fritzing, en la Figura 4 se ilustra el diagrama de conexiones.



1) Conexiones del CD4051

El circuito CD4051 se alimentó a 5 volts por medio del pin V_{DD} , el pin $COM\ in/out$ se conectó igual a 5 volts, este voltaje será el que habrá en la salida seleccionada. Los pines INH , V_{EE} y V_{SS} se conectaron a GND. Los pines de control se configuraron de la siguiente manera: $A \rightarrow D8$, $B \rightarrow D9$, $C \rightarrow D10$. Finalmente, la salida $X0$ se conectó a la resistencia de $10\text{ K}\Omega$ y la salida $X1$ a la resistencia de $1\text{ M}\Omega$.

2) Conexiones del circuito integrado 555

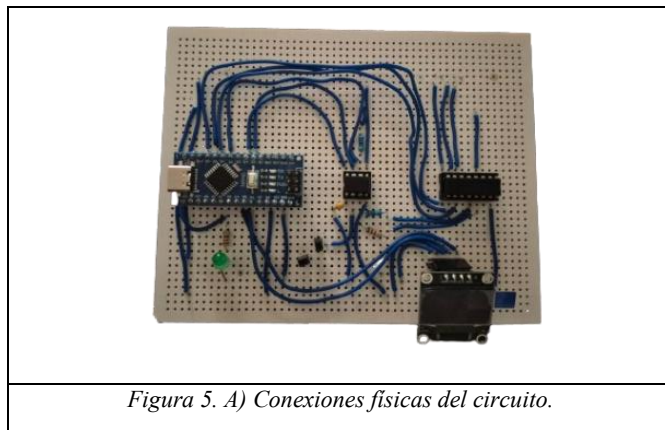
El circuito integrado 555 se alimentó a 5 volts por medio del pin 8, el pin 1 se conectó a tierra, el pin 2 identificado como *trigger* se conectó a una resistencia de Pull-Up de $4.7\text{ K}\Omega$, para evitar falsos disparos y que el pin estuviese flotando, además este mismo pin se conectó a pin D8 del Arduino. La salida correspondiente al pin 3 se conectó al pin D3 del Arduino, el pin 4 llamado *reset* se conectó al D4, el pin 5 encargado del control de voltaje se conectó a un capacitor de $10\text{ }\mu\text{F}$, el pin 7 se conectó a las resistencias de $10\text{ K}\Omega$ y $1\text{ M}\Omega$,

3) Conexiones del Arduino Nano

Las conexiones adicionales que se realizaron con el Arduino Nano fueron las alimentaciones del circuito por medio del pin de 5 V y el pin de GND , es importante mencionar que todo el circuito se alimentó a 5 V . El pin 12 se conectó a una resistencia de $330\text{ }\Omega$ que va conectada a un led.

B. Construcción de circuito físico.

El circuito físico se construyó inicialmente en una protoboard para realizar pruebas y ajustar el valor de las resistencias para generar una constante de tiempo donde abarcara un mayor rango de medición. Para el diseño final, el circuito se construyó en una placa perforada, utilizando cable del calibre 28 se realizaron todas las conexiones, el resultado final se observa en la Figura 5.



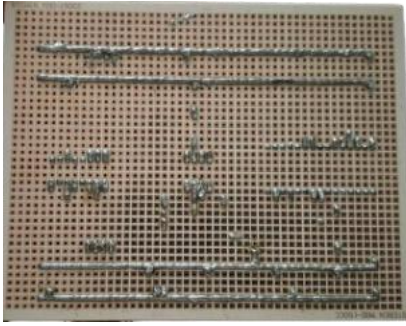


Figura 6. Pistas de conexiones del circuito.

C. Estructura del código.

1) Librerías implementadas

Las librerías utilizadas fueron 3, las cuales se encargaron de desarrollar la comunicación de la pantalla oled ssd1306 con el Arduino.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

2) Configuración de la pantalla oled ssd1306

Se definieron las dimensiones de la pantalla oled ssd1306 (ancho y alto), además de definir el *reset* como falso (-1) y su dirección de memoria, normalmente la dirección de memoria suele ser 0x3C. Finalmente se le mandan los parámetros definidos con anterioridad a la función `Adafruit_SSD1306 display()` para inicializar la pantalla de manera correcta.

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH,
SCREEN_HEIGHT, &Wire, OLED_RESET);
```

3) Declaración de variables

Se utilizó la variable `R1` para guardar el valor de la resistencia física, `Duracion` esta variable guarda la cantidad de tiempo que dura la carga del capacitor, `t_inicio` guarda el tiempo de inicio de la carga del capacitor, `t_final` guarda el tiempo final de la carga del capacitor, `pulso_leido` sirve de bandera para habilitar cuando el pulso ha sido leído, `disparo` bandera que se encarga de habilitar o deshabilitar el pulso de disparo del Arduino, `rango1` bandera para el rango de medición menor, `rango2` bandera para el rango de medición mayor,

`TIMEOUT_MS` se encarga de guardar una constante de tiempo para el tiempo de descarga del capacitor.

```
float R1;
unsigned long duracion;
volatile unsigned long t_inicio = 0;
volatile unsigned long t_final = 0;
volatile bool pulso_leido = false;
volatile bool disparo = 0;
volatile bool rango1 = 1;
volatile bool rango2 = 0;
const unsigned long TIMEOUT_MS = 5000;
```

4) Declaración de pines del multiplexor CD4051

Se declararon los pines 8, 9 y 10 para controlar los pines de control A, B y C.

```
const int pinA = 8;
const int pinB = 9;
const int pinC = 10;
```

5) Inicio de comunicación serial

Se inicializo la comunicación serial con una velocidad serial de 9600 baudios, se utilizó un retraso de tiempo para que el puerto esté listo, finalmente se limpia el buffer de salida.

```
Serial.begin(9600);
while (!Serial);
delay(500);
Serial.flush();
```

6) Inicialización de pantalla Oled ssd1306

Se inicializo la pantalla oled, con la dirección de memoria declarada anteriormente, además se le asigno el tamaño y color de texto.

```
if(display.begin(SSD1306_SWITCHCAPVCC,
SCREEN_ADDRESS)) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.display();
}
```

7) Declaración de pines

Los pines utilizados fueron los siguientes:

```
pinMode(12, OUTPUT); // Pin LED
pinMode(3, INPUT); // Pin 3 (Salida del 555)
pinMode(7, OUTPUT); // Pin Trigger
pinMode(4, OUTPUT); // Pin Reset
pinMode(pinA, OUTPUT); // Pin A CD4051
```



```
pinMode(pinB, OUTPUT); // Pin B CD4051
pinMode(pinC, OUTPUT); // Pin C CD4051
```

8) Activación de pines

El pin de *reset* que se pone en alto para que el circuito integrado 555 se habilite y el pin de disparo se pone en estado alto para que no esté flotando o mandando disparos falsos.

```
digitalWrite(4, HIGH); // Ponemos el RESET del
555 en bajo
digitalWrite(7, HIGH); // Ponemos el TRIGGER
en alto
```

9) Función de interrupción

Se utilizó la función `attachInterrupt` configurada para el pin 3, esta función se encarga de detectar cuando se manifiesta un cambio de estado en el pin 3 y manda llamar la función `cronometro`. También se manda llamar la función `cambiarRango` configurada para el primer rango de medición.

```
attachInterrupt(digitalPinToInterrupt(3),
cronometro, CHANGE);
cambiarRango(0);
```

10) Bloque de disparo para el circuito integrado 555

Este bloque de disparo se encarga de verificar si la bandera `disparo` es falsa y si el pin 3 se encuentra en estado bajo, después se tiene un retraso de tiempo de 5000 milisegundos para descargar el capacitor, el pin 3 se manda a estado bajo, se pone un retardo de 3 milisegundos, después se vuelve a poner el pin 3 en alto. En el serial del Arduino se manda imprimir el mensaje de calculando al igual que en la pantalla Oled ssd1306.

```
if(disparo == 0 && digitalRead(3) == LOW){
delay(5000);
digitalWrite(7, LOW);
delayMicroseconds(3);
digitalWrite(7, HIGH);
Serial.println("Calculando...");
imprimir("Calculando", 30, 8);
}
```

11) Bloque de tiempo actual

Este bloque se encarga de contar el tiempo de carga del capacitor, si este es mayor al intervalo de tiempo asignado, entonces manda llamar la función `encarga` del cambio de rango. Su funcionamiento se basa en comparar el estado en alto del pin

3, además compara la diferencia del tiempo inicial y el tiempo actual con el intervalo de tiempo de referencia, si la condición es cierta se imprime un mensaje en el serial y la pantalla oled indicando el cambio de rango, además se manda reiniciar el circuito integrado 555, por lo tanto, la variable que guarda el tiempo inicial se reinicia, la bandera del pulso leído y disparo se hacen falsas, finalmente la bandera del rango 1 se deshabilita y la bandera del rango 2 se habilita mandando llamar a la función `cambiarRango`.

```
if (digitalRead(3) == HIGH && (micros() -
t_inicio) > (TIMEOUT_MS * 1000ULL)) {
```

```
Serial.println("Cambiando rango->");
imprimir("Cambiando rango ->", 10, 8);
```

```
digitalWrite(4, LOW);
delay(100);
digitalWrite(4, HIGH);
```

```
t_inicio = 0;
pulso_leido = false;
disparo = 0;
rango1=0;
rango2=1;
cambiarRango(1);
```

```
}
```

12) Bloque de cálculo de capacitancia

a) Delta de tiempo

El bloque del cálculo de capacitancia comienza verificando que la bandera de `pulso_leido` sea verdadera, después se compara que el tiempo final sea mayor al tiempo inicial para obtener una medida de tiempo correcta, se realiza la diferencia entre el tiempo final e inicial y se guarda en la variable `duracion`; si el tiempo final no es mayor que el tiempo inicial significa que ocurrió un falso disparo, entonces se manda restablecer el circuito integrado 555 por medio del pin de *reset*, la bandera del disparo se vuelve verdadera y el tiempo de inicio se pone en 0 para comenzar una cuenta nueva.

```
if (pulso_leido) {
```

```
if(t_final > t_inicio){
```

```
duracion = t_final - t_inicio;
```

```
}else {
```

```
digitalWrite(4, LOW);
```




```

delay(100);
digitalWrite(4, HIGH);
disparo = 1;
t_inicio = 0;
}

```

b) Cálculo de capacitancia

El bloque inicia comparando si la duración es mayor a 30 milisegundos y menos 12×10^6 milisegundos, esto para verificar que este en un rango correcto para el primer rango de medición y no se obtenga tanto error en la medición. Esta condición se verifica para ambos rangos. Por medio de la Ecuación 2 se determinó el valor de la capacitancia, cabe mencionar que la variable *duracion* ya cuenta con el delta de tiempo final menos tiempo inicial, esta delta se calcula con la función *cronometro* que se explicará más adelante. Después se verifica si el valor de la capacitancia es menor a 0.1, si esto se cumple, entonces el valor calculado de la capacitancia se multiplica por 1000 para mostrar el resultado en nano faradios, el resultado se imprime en el serial y la pantalla oled. Si el valor calculado de la capacitancia es mayor a 0.1, entonces se imprime el valor calculado en el serial y pantalla oled.

```

if (duracion > 30 && duracion < 12000000 &&
rango1 == 1 || duracion > 30 && duracion <
12000000 && rango2 == 1) {

    float capacitancia = duracion / (1.1 * R1);
    if (capacitancia < 0.1) {
        Serial.print("C = ");
        Serial.print(capacitancia * 1000.0, 2);
        Serial.println(" nF");
        imprimirC(capacitancia * 1000.0, "nF");
    } else {

        Serial.print("C = ");
        Serial.print(capacitancia, 2);
        Serial.println(" uF");
        imprimirC(capacitancia, "uF");
    }
}

```

c) Capacitancias parasitarias

Cuando el primer rango de medición se encuentra activado y la duración es menor a los 30 milisegundos, entonces se imprime el valor de una capacitancia de 0, ya que existen capacitancias parasitarias en el circuito, estas suelen ser de un valor muy pequeño.

```

} else if (duracion < 30 && rango1 == 1) {
    Serial.println("C = 0");
    imprimirC(0, "uF");
}

```

d) Cambio al rango de medición menor.

Finalmente, en este bloque del cálculo de capacitancia se verifica si el rango mayor de medición es verdadero y la variable *duracion* se encuentra entre el rango de los 15000 y 10 milisegundos, si esta condición es verdadera, entonces el rango de medición menor se activa y el mayor se desactiva, además de mandar a llamar la función *cambiarRango*. Este cambio rango se realiza para que el circuito pueda medir de nuevo capacitores de menor valor, ya que, si se miden con el rango mayor, la medición tendrá mucho error. En la parte final de este bloque la bandera *pulso_leido* y *disparo* se vuelven falsas.

```

} else if (duracion < 15000 && duracion > 10 &&
rango2 == 1) {
    Serial.println("Cambiando rango <-");
    imprimir("Cambiando rango <-", 10, 8);
    rango1=1;
    rango2=0;
    cambiarRango(0);
}

pulso_leido = false;
disparo = 0;
}

```

13) Función cronometro

Esta fusión se encarga de verificar si el pin 3 del Arduino que está conectado al pin de salida del circuito integrado 555 cambia su estado, cuando el estado se encuentra en alto se enciende un led testigo que se encuentra conectado al pin 12 del Arduino, además se guarda el tiempo en ese instante donde la salida paso de estado bajo a alto y la bandera *disparo* se hace verdadera. Si el pin 3 se encuentra en estado bajo, se guarda el tiempo en que sucedió la transición de estado, el led testigo se apaga, la bandera *pulso_leido* se vuelve verdadera y la bandera *disparo* se vuelve falsa.

```

void cronometro() {

    if (digitalRead(3) == HIGH) {
        digitalWrite(12, HIGH);
        t_inicio = micros();
        disparo = 1;
    } else {

```

```
t_final = micros();
digitalWrite(12, LOW);
pulso_leido = true;
disparo = 0;
```

```
}
```

14) Función autorango

Esta función se encarga de asignar un numero binario a cada uno de los pines de control del multiplexor, por lo tanto, cuando los pines de control formen el número de cualquier salida ($X_0, X_1, X_2, \dots$) esta se seleccionará. En esta función también se modifica el valor de la resistencia de carga dependiendo que rango se asigne.

```
void cambiarRango(int rango) {

    digitalWrite(8, bitRead(rango, 0));
    digitalWrite(9, bitRead(rango, 1));
    digitalWrite(10, bitRead(rango, 2));

    if(rango == 0) R1 = 1.046e6 + 420;
    if(rango == 1) R1 = 0.982e3 + 420;

}
```

15) Función de impresión de mensaje en pantalla oled

La función de impresión tiene el objetivo de imprimir el mensaje de calculando en la pantalla oled, se le mandan parámetros como el tamaño de letra, posición donde se imprimirá y el mensaje, además el comando `display.display()` se encarga de actualizar el mensaje que se le manda a la pantalla oled.

```
void imprimir(String mensaje, int x, int y) {

    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(x, y);
    display.print(mensaje);
    display.display();

}
```

16) Función de impresión del valor de capacitancia

Esta función se encarga de imprimir en la pantalla oled el valor de la capacitancia calculada, se imprime un recuadro

negro en las coordenadas donde se pondrá el valor para que cada interacción se esté limpiando y no se empalme con el valor nuevo.

```
void imprimirC(float valor, String unidad) {

    String valorTexto = String(valor);
    int longitudTotal = 3 + valorTexto.length() +
    unidad.length();
    display.setTextSize(1);
    display.setCursor(0, 30);
    display.fillRect(0, 30, 128, 8,
    SSD1306_BLACK);
    display.setCursor(64 - 6 * longitudTotal / 2,
    30);
    display.print("C= ");
    display.print(valorTexto);
    display.print(unidad);
    display.display();

}
```

D. Calibración del circuito

Para obtener unas mediciones más cercanas a los valores reales, los primero que se realizó fue medir el valor de las resistencias de carga de ambos rangos como se observa en la Figura 7.



Los valores reales obtenidos de las resistencias se declararon en el código para obtener un cálculo más exacto. Como ya se había mencionado anteriormente el multiplexor proporciona una resistencia interna que varía desde los 400Ω a 520Ω con una alimentación de $5 V$ a una temperatura de $25^\circ C$ [5], pero al ser un caso en condiciones específicas, esto nos proporciona un cierto error al realizar las mediciones con el circuito en condiciones distintas. Para obtener un valor más real de la resistencia interna del multiplexor se observó la curva de carga de 3 capacitores distintos, por medio del modo cursor con el que

cuenta el osciloscopio, se midió el tiempo de inicio y final de la curva de carga y se determinó la resistencia interna del multiplexor de acuerdo con la Ecuación 3.

$$R_{ON} = \frac{T}{1.1 \times C_1} - R_1 \quad (3)$$

En la Figura 8 se observan las mediciones de la curva de carga para un capacitor de $333 \mu f$ por medio del osciloscopio.

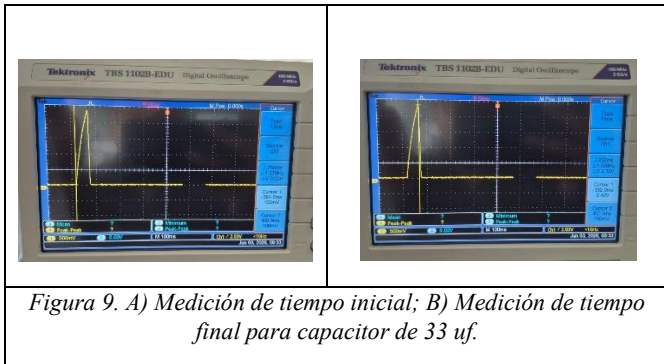


De acuerdo con la Figura 8, se obtuvo un tiempo inicial de $0.9 s$ y un tiempo final de $1.420 s$. Se desarrollo el cálculo en base a la Ecuación 3 y se obtuvo lo siguiente:

$$R_{ON} = \frac{1.420[s] - 0.9 [s]}{1.1 \times 333 \mu f} - 0.982k\Omega$$

$$R_{ON} = 437.601 \Omega$$

En la Figura 9 se observan las mediciones de la curva de carga para un capacitor de $33 \mu f$ por medio del osciloscopio.

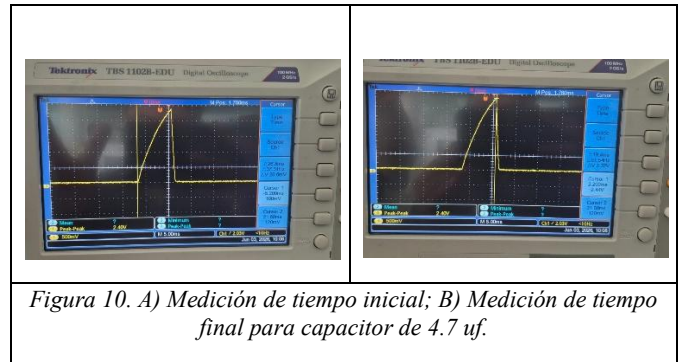


De acuerdo con la Figura 9, se obtuvo un tiempo inicial de $-384 ms$ y un tiempo final de $-332 ms$. Se desarrollo el cálculo en base a la Ecuación 3 y se obtuvo lo siguiente:

$$R_{ON} = \frac{-332[ms] - (-384 [ms])}{1.1 \times 33 \mu f} - 0.982k\Omega$$

$$R_{ON} = 450.5068 \Omega$$

En la Figura 10 se observan las mediciones de la curva de carga para un capacitor de $4.7 \mu f$ por medio del osciloscopio.



De acuerdo con la Figura 10, se obtuvo un tiempo inicial de $-5.2 ms$ y un tiempo final de $2.2 ms$. Se desarrollo el cálculo en base a la Ecuación 3 y se obtuvo lo siguiente:

$$R_{ON} = \frac{2.2[ms] - (-5.2 [ms])}{1.1 \times 4.7 \mu f} - 0.982k\Omega$$

$$R_{ON} = 449.334 \Omega$$

En base a los cálculos anteriores, se determinó una resistencia interna (R_{ON}) promedio de la siguiente manera:

$$R_{average} = \frac{437.601 + 450.5068 + 449.334}{3}$$

$$R_{average} = 445.81 \Omega$$

La resistencia promedio calculada se utilizó en el código de Arduino con el objetivo de obtener valores con un menor margen de error en cada medición.



VI. RESULTADOS

A. Medición de capacitores con Arduino nano.

Se realizaron 5 mediciones calculadas con el Arduino Nano, los valores de los capacitores fueron los siguientes:

$$33 \mu f, 0.1 \mu f, 333 \mu f, 1nf, 4.7 \mu f$$

En la Tabla 1 se muestran los valores obtenidos por el circuito en cada uno de los valores de capacitancia.

Tabla 1. Valores calculados por el Arduino Nano.

Valor ideal	Valor medido
333 μf	335.12 μf
33 μf	34.32 μf
4.7 μf	4.68 μf
0.1 μf	0.12 μf
1 nf	1.07 nf

Se aprecia que no existe mucho error entre el valor ideal y el valor calculado por el Arduino Nano, para saber cuánto porcentaje de error tiene cada una de las mediciones se utilizó la Ecuación 4.

$$E_{\%} = 100 \times \left(\left| \frac{\text{ValorTeórico} - \text{ValorExperimental}}{\text{ValorTeórico}} \right| \right) \quad (4)$$

Se calculo el porcentaje de error para cada medición y se generó un promedio para saber con cuanto error es que cuenta el circuito.

El porcentaje de error para 333 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{333 \mu f - 335.12 \mu f}{333 \mu f} \right| \right)$$

$$E_{\%} = 0.636 \%$$

El porcentaje de error para 33 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{33 \mu f - 34.32 \mu f}{33 \mu f} \right| \right)$$

$$E_{\%} = 4 \%$$

El porcentaje de error para 4.7 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{4.7 \mu f - 4.68 \mu f}{4.7 \mu f} \right| \right)$$

$$E_{\%} = 0.425 \%$$

El porcentaje de error para 0.1 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{0.1 \mu f - 0.12 \mu f}{0.1 \mu f} \right| \right)$$

$$E_{\%} = 20 \%$$

El porcentaje de error para 1 nf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{1 nf - 1.07 nf}{1 nf} \right| \right)$$

$$E_{\%} = 7 \%$$

Ya obtenidos los porcentajes de error individual de cada medición, se calculó el porcentaje de error general de la siguiente manera:

$$E_{G\%} = \frac{0.636 + 4 + 20 + 7 + 0.425}{5}$$

$$E_{G\%} = 6.412 \%$$

El porcentaje de error general fue menor al 10 %, por lo tanto, se llega a la conclusión de que el circuito construido funciona de manera correcta y con un margen de error tolerante.

B. Medición de capacitores con multímetro.

Se realizaron las mediciones de los mismos capacitores, pero ahora con un multímetro ASTROAI DM6000AR obteniendo las mediciones que se ilustran en la Tabla 2.

Tabla 2. Valores medidos por el multímetro.

Valor ideal	Valor medido
333 μf	342.5 μf
33 μf	36.17 μf
4.7 μf	5.03 μf
0.1 μf	0.11 μf
1 nf	0.98 nf

En este caso, también se utilizó la Ecuación 4 para calcular el porcentaje de error individual de cada medición.

El porcentaje de error para 333 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{333 \mu f - 342.5 \mu f}{333 \mu f} \right| \right)$$

$$E_{\%} = 2.852\%$$

El porcentaje de error para 33 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{33 \mu f - 36.17 \mu f}{33 \mu f} \right| \right)$$

$$E_{\%} = 9.606\%$$

El porcentaje de error para 4.7 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{4.7 \mu f - 5.03 \mu f}{4.7 \mu f} \right| \right)$$

$$E_{\%} = 7.021\%$$

El porcentaje de error para 0.1 μf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{0.1 \mu f - 0.11 \mu f}{0.1 \mu f} \right| \right)$$

$$E_{\%} = 10\%$$

El porcentaje de error para 1 nf fue el siguiente:

$$E_{\%} = 100 \times \left(\left| \frac{1 nf - 0.98 \mu f}{1 nf} \right| \right)$$

$$E_{\%} = 2\%$$

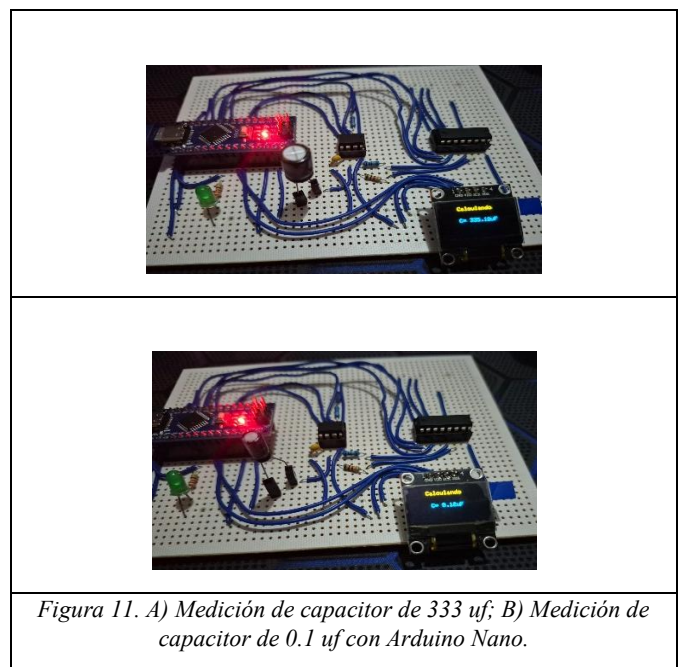
Ya obtenidos los porcentajes de error individual de cada medición con el multímetro, se calculó el porcentaje de error general de la siguiente manera:

$$E_{G\%} = \frac{2.852 + 9.606 + 7.021 + 10 + 2}{5}$$

$$E_{G\%} = 6.295\%$$

C. Comparativa de mediciones

El error que se obtuvo al utilizar el Arduino Nano para calcular la capacitancia fue muy parecido al error que se obtuvo al realizar las mediciones con el multímetro, ambos errores son menor res al 10 % y existe una diferencia entre estos de 0.117 la cual es muy poca. Es importante mencionar que el circuito desarrollado en este proyecto se puede calibrar de mejor manera al realizar más mediciones con distintos valores de capacitores y verificar su curva de carga en el osciloscopio para obtener su periodo de tiempo y así encontrar una resistencia interna del multiplexor más exacta para brindar un menor margen de error. En la Figura 11 y 12 se ilustran algunas de las mediciones realizadas con el circuito y el multímetro.



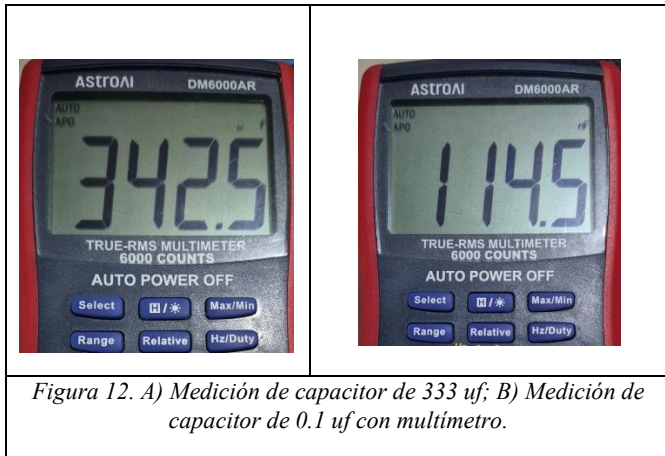


Figura 12. A) Medición de capacitor de 333 uf; B) Medición de capacitor de 0.1 uf con multimetro.

VII. COMENTARIOS

Los comentarios personales durante este proyecto se basan más en los puntos de mejora que se observaron al estar probando el funcionamiento del circuito, uno de estos sería ampliar el rango de medición, el rango de 0.15 nf a $1000\text{ }\mu\text{f}$ es considerablemente amplio, este se puede ampliar y para ello habría que realizar más pruebas de medición y obtener el periodo de la curva de carga por medio de un osciloscopio e implementar el cálculo desarrollado en el apartado de resultados, esto brindaría una resistencia interna del multiplexor más exacta a las condiciones de trabajo actuales y con este valor se podría determinar un tercer rango de medición para capacitores mayores a $1000\text{ }\mu\text{f}$, habría que calcular la constante de carga para este nuevo rango y en base a eso seleccionar una resistencia de carga que en complemento de la resistencia interna del multiplexor cumplan con el periodo de carga establecido.

Otro comentario de mejora es implementar otro microcontrolador con una frecuencia mayor a la del Arduino Nano, en este caso sería una placa ESP32 este microcontrolador permitiría medir capacitores más pequeños al de 0.15 nf . Es importante mencionar que la medición de los pulsos de carga en capacitores muy pequeños suele ser muy rápida en este caso el Arduino Nano llega a tener un margen considerable de error con los capacitores menores a 0.15 nf .

Finalmente, los aprendizajes que obtuve durante el desarrollo de este proyecto se basaron más en el funcionamiento del multiplexor, ya que no tenía experiencia trabajando con uno de ellos, además de conocer cómo es que se manejan los pines de control, salida y entrada. También considerar la resistencia interna que contienen este tipo de circuitos integrados. El funcionamiento de circuito integrado 555 ya lo había adquirido en semestres anteriores, pero la nueva experiencia que obtuve fue realizar una manipulación de este tipo de circuitos con un microcontrolador como Arduino, en el aspecto de la programación aprendí el comportamiento de funciones de

tiempo y como adaptarlas para estar monitoreando el tiempo actual para controlar la carga y descarga de los capacitores.

VIII. CONCLUSIONES

Realizar un circuito de medición suele ser algo complejo, ya que se tienen que considerar ciertos parámetros, en este caso se consideró el valor de las resistencias de carga para los capacitores, la resistencia interna del multiplexor, el voltaje de alimentación para el circuito y los tiempos de carga y descarga de los capacitores, esto fue fundamental para calibrar el circuito de medición y obtener un margen de referencia, si alguno de estos parámetros no es el correcto, se obtendrá una medición con un margen de error considerable. Otro punto al que se llegó fue conocer el funcionamiento de cada uno de los circuitos integrados con los que se va a trabajar, ya que estos suelen cambiar sus características dependiendo el voltaje de alimentación que se le suministre y la temperatura ambiente a la que estén trabajando, por otro lado en los microcontroladores es importante conocer la frecuencia interna de reloj con la que trabajan, ya que para algunas aplicaciones de medición una frecuencia como la del Arduino no suele ser suficiente para obtener buenos resultados.

El método utilizado para calcular la capacitancia en este proyecto fue por medio de una constante de tiempo, como se mencionó en el apartado de resultados, el error obtenido no superó el 10 % y fue muy cercano al error de medición que se obtiene con un multimetro, gracias a esto se puede decir que este método es uno de los mejores para realizar la medición de capacitancia, el único detalle es que depende mucho de la calibración y los rangos de medición, ya que con un único valor de resistencia no se pueden medir ciertos valores de capacitores porque estos tendrían un margen de error muy grande.

IX. BIBLIOGRAFÍAS

[1] Instructables, "Capacitance meter with arduino and 555 timer," *Instructables*, [En línea]. Disponible: <https://www.instructables.com/Capacitance-meter-with-arduino-and-555-timer/>. [Accedido: 4-Mar-2026].

[2] Instructables, "Medidor de capacitancia con el arduino y el op-amp," *Instructables*, [En línea]. Disponible: <https://www.instructables.com/Medidor-de-capacitancia-con-el-arduino-y-el-op-amp/>. [Accedido: 4-MAR-2026].

[3] El rincón del vago, "Como hacer un medidor de capacitancia con arduino," *YouTube*, 15-mar-2021 [Video en línea]. Disponible: <https://www.youtube.com/watch?v=7crhnqi5D0M>. [Accedido: 26-Abr-2026].



[4] Texas Instruments, "LM555 Timer Datasheet," Hoja de datos de producto, SNAS548D, feb. 2000 [Revisada ene. 2015].

[5] Fairchild Semiconductor, "CD4051BC-CD4052BC-CD4053BC Single 8-Channel Analog Multiplexer/Demultiplexer," Hoja de datos de producto, nov. 1983 [Revisada ago. 2000].